Constraint-Programmed Initial Sizing of Analog Operational Amplifiers

Inga Abel, Maximilian Neuner and Helmut Graeb
Chair of Electronic Design Automation, Technical University of Munich, 80333 München, Germany
{inga.abel,maximilian.neuner,graeb}@tum.de

***Abstract*—This paper presents a new method to automate the initial sizing of operational amplifiers. The method emulates the manual design procedure. The sizing task is formulated as a constraint programming problem. Two new algorithms are introduced: First, a hierarchical structural analysis of functional blocks that automatically sets up the analytical equations for the sizing. And second, a heuristic to guide branching process of a constraint programming solver. We achieve a reproduction of the manual proceeding of designers and at the same time an automation of the set-up of the design problem, reducing the set-up effort from months to seconds. 16 topologies, including different variants of two-stage, folded-cascode, telescopic and symmetrical op-amps, are considered at this time. In this paper, experimental results for a two-stage op-amp and a folded-cascode op-amp are presented to show the effectiveness of the approach.**

I. Introduction

From the three steps of the analog design process, i.e., structure design, sizing, layout design, this paper focuses on the sizing step. Sizing refers to determining the circuit design parameters, e.g., transistor widths/lengths of CMOS circuits, such that the performance requirements i.e. given lower or upper bounds that must be kept, e.g. minimum gain, maximum slew rate, are fulfilled. The sizing step usually comprises two sub-steps, initial sizing and optimization. This paper deals with initial sizing.

Initial sizing: Initial sizing starts from the circuit structure without having any values of the design parameters to start from. It aims at finding a first, i.e., initial sizing of the circuit considering the performance specification. Traditionally, initial sizing is done manually and by applying analytical equations of the circuit behavior. This type of sizing is also called equation-based sizing as distinguished to simulation-based sizing which applies SPICE-like numerical simulation.

Equation-based sizing: Early design automation tools for analog sizing were equation-based. They implemented fixed design plans for each considered topology [1]–[4]. These tools suffer disadvantages: The programming of a circuit-specific design plan must be done for each individual circuit topology and takes an extremely long time. And the fixed design plan prevents optimization and design space exploration. As a remedy, optimization-based techniques came up [5]–[8], while maintaining analytical formulas of the design plan-based approaches. Graphical support for an interactive design through the design equations is another approach [9]. To support the programming of topology-dependent design plans, symbolic analysis techniques were developed to automatically create analytical performance models [10]–[12]. Yet, the growing complexity of transistor models in deep submicron technologies led to a large gap in accuracy of analytical equations for the circuit behavior.

Simulation-based sizing: To gain more topology independence and maintain more accuracy in behavior models, simulation-based numerical optimization methods for circuit sizing have been developed [13]–[24]. The approaches [15] and [16] have been transferred into commercial design suites of the companies MunEDA [25] and Cadence, respectively. These methods also include advanced design aspects like manufacturing variation and yield optimization, aging or Pareto optimization. It is important to note that simulation-based sizing needs an initial sizing to start from. The chosen initial sizing has influence on the convergence of the numerical simulation. Therefore, it should provide some reasonable performance values.

Industrial practice in analog sizing: The industrial practice of analog sizing is manual and equation-based, followed by simulation, corner-case simulation and Monte-Carlo analysis for verification. Although simulation-based, optimization-based sizing provides topology independence, potentially better accuracy and results, designers prefer the traditional way of initial sizing. There may be two reasons for this: One is the lack of physics orientation and lack of insight that comes with numerical methods. The other is the huge effort to set up the numerical sizing and the requirement of additional knowledge to guide the numerical optimization process.

Contribution of this paper: This paper presents a new attempt for a sizing process that goes along with the designers' wish for analytical, physics-related design equations. The contribution comprises two main parts: First, a new method to automatically set up the design equations from an analysis of the circuit netlist is presented. The netlist's internal hierarchy is decomposed into single transistors, transistor pairings and basic functional blocks. Analytical design equations from analog design books for the transistor pairings and functional blocks have been collected in a library. Once an op-amp topology has been decomposed, the related design equations are automatically instantiated. These equations consist of equalities and inequalities between parameters and performance features, which describe the behavioral relation, constraints and performance specification.

Second, these equations are fed into an optimization method

978-1-5386-6648-7/19/$31.00 ©2019 IEEE
DOI 10.1109/ICCD46524.2019.00065

from the artificial intelligence area, namely constraint programming [26]–[30]. We decided for constraint programming because it does not require a specific type of performance function as, e.g, posynomial models in [6], and as it is suitable for the combinatorial character of analog sizing due to the manufacturing-induced discretized value range of transistor geometries. Constraint programming provides the required branch-and-bound techniques for this type of problems and does not approximate the sizing task by a continuous optimization approach with subsequent snap-to-grid as e.g. in [5]. The branching process of constraint programming requires a careful guidance to be successful. We present a method which analyzes the circuit netlist and derives a heuristic ordering for the branching process for the op-amp design equations.

The setup of design equations together with a constraint programming tool replaces the effort of a manual setup that takes months by an automatic procedure that takes seconds, while rigorously following the practical proceeding of sizing. The method is verified for 16 different op-amp topologies, including different variants of two-stage op-amps, folded-cascode op-amps, and symmetrical OTAs. More types of op-amps (three-stage, rail-to-rail, etc.) will be included in the future.

Discussion of limitations:

Computational complexity: Branch-and-bound problems usually do not scale and are inadequate for complex problems. The op-amp sizing task's complexity is in a range that can be handled well with constraint programming. A CPU time issue may arise from inadequate ordering in the branching process. This is mitigated with the presented ordering algorithm. The resulting setup and solution times are within a minute regardless of the op-amp complexity.

Convergence: The ordering of the branching process is important for the convergence of constraint programming. The presented ordering approach according to the biasing flow is the main contributor. Additional aspects for ordering towards better convergence rate are under investigation.

Accuracy of behavior modeling: The paper does not compare with simulation-based, optimization-based approaches. It aims at automating the equation-based sizing process that designers typically use and increase the spectrum of op-amp topologies that can be handled. The used equations refer to the design equations in literature, which are known to be deviating from SPICE accuracy. Hence, an optimization usually will follow after initial sizing. The achieved gain is in the automation of the setup and in an optimization within the obtained design degrees of freedom.

Extension to other circuits: The method is specific to op-amps with a certain set of functional blocks. If a new functional block within an op-amp arises, its design equations have to be added. The main limitation at this point are feedback loops within the circuit, which are not yet considered. Current work is on this topic. We decided to start our research with op-amps, because they are frequently used in analog and mixed-signals circuits with different circuitries. However, they are still designed separately and manually, according to the specification

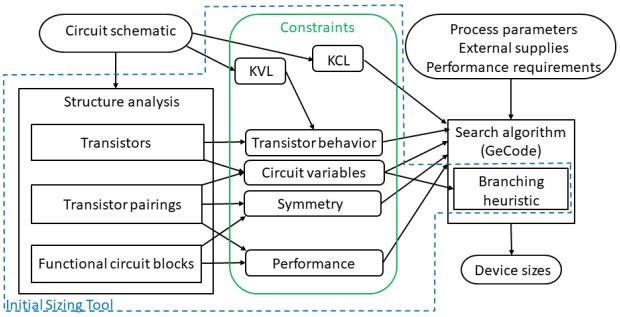

Figure 1: Overview of the initial sizing tool

derived from their function in the overall circuit. Other circuit classes are future work.

Paper structure: The paper is organized as follows: Sec. II presents the setup of the initial sizing task as a constraint program, with an emphasis of the possibility to break down all design equations to different circuit hierarchy levels. Sec. III presents the new functional circuit block analysis. Sec. IV presents the new branching heuristic. Experimental results on two circuits are given in Sec. V. It includes a comparison of the newly developed initial sizing tool with other state-of-art equation-based sizing tools, the latest from 2001 [31]. More recent attempts to closely follow the manual analog design process have not been made. The paper ends with a conclusion (Sec. VI).

II. Constraint Formulation of the Initial Sizing Task

The presented initial sizing tool (Fig. 1) is based on the open-source constraint programming library GeCode [32]. The analog sizing task is formulated as a Constraint-Optimization-Problem (COP, see Appendix A). The optimization target, i.e. maximizing performance safety margins, is described by equation (5) in Sec. II-A. Except for the problem specification, all parts of the problem setup and solution have been automated. Circuit schematic and problem specification, i.e., technology parameters, external supplies and performance requirements, are input. The initial sizing tool automatically determines the variables, equality and inequality constraints given in the middle of Fig. 1. The variables refer to:

- the circuit performances $\mathbf{f}$, (3),
- the transistor geometries $\mathbf{x}$, (7),
- the drain-source currents $\mathbf{i}_{DS}$, (9),
- and node voltages $\mathbf{v}_N$, (12).

The constraints refer to:

- Kirchhoff's Current Law, (13),
- behavioral description (16) and operating conditions (17) of transistors,
- constraints to ensure the proper functionality of functional circuit blocks, (19) - (20),
- performance models (21) derived from functional circuit blocks and performance constraints (22).

The variables and constraints are directly derived from the circuit netlist, which is the case for KCL and KVL, or generated by a hierarchical structural analysis of the circuit netlist. The structural analysis is classified into three hierarchical levels. On the lowest level, single transistors are analyzed and the variables of the sizing problem are extracted. Also constraints on the transistor behavior are generated. On the next level, basic transistor pairings are examined and dependent design parameters and symmetry constraints are derived. On the next hierarchy level, functional blocks of the circuit are analyzed and constraints regarding the op-amp performance requirements are set up. To identify functional blocks of an op-amp, a new algorithm was developed. It is described in Sec. III.

The automatically generated variables and constraint are handed to the constraint solver GeCode. To explore the solution space, a Restart-Based-Search (RBS) is used. This is an intelligent search algorithm provided by GeCode, which takes previously executed searches into account. An RBS restarts a Branch-And-Bound algorithm (BAB, see Appendix A) if no solution was found after a given number of iterations. The restarted BAB uses advanced methods to restrict the search space through constraint manipulations. Still, the CPU time for analog sizing is only feasible if a sophisticated branching method guides the RBS. This method is explained in Sec. IV.

In the following, the problem specification and the set-up of the variables and constraints is described in detail.

A. *Specification of the Problem*

The following input is required:

Circuit schematic: The initial sizing starts from a given schematic, as for instance the ones in Figs. 2 and 3, in shape of a netlist.

External supplies: Supply voltages, bias voltages and bias currents set the boundary conditions of the sizing problem:

$$\Phi = \{VDD, VSS, \mathbf{i}_{Bias}, \mathbf{v}_{Bias}\} \tag{1}$$

Technology parameters: From the process development kit of the target manufacturing process technology, a subset of device parameter values are required to describe the device behavior, e.g. for transistors:

$$\Lambda := \{C_{ox}, t_{ox}, v_{th_{n/p}}, \lambda_{n/p}, ...\} \tag{2}$$

Performance features and performance requirements: The performance features $\mathbf{f}$ are specified as objectives of the sizing process. Examples are the open-loop gain, the power consumption, or the slew rate:

$$\mathbf{f}^T = [f_1, f_2, ..., f_{n_f}] \tag{3}$$

At this moment, the initial sizing tool supports twelve different performance features (Table II). For each performance feature f_i, either a minimally or maximally required value $f_{B,i}$ that has to be achieved, can be specified:

$$\mathbf{f}_B^T = [f_{B,1}, f_{B,2}, ..., f_{B,n_{f_B}}], \quad f_i \geq f_{B,i} \text{ or } f_i \leq f_{B,i} \tag{4}$$

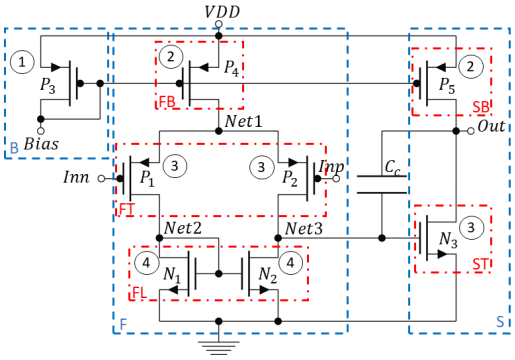

Figure 2: Schematic of a two-stage op-amp

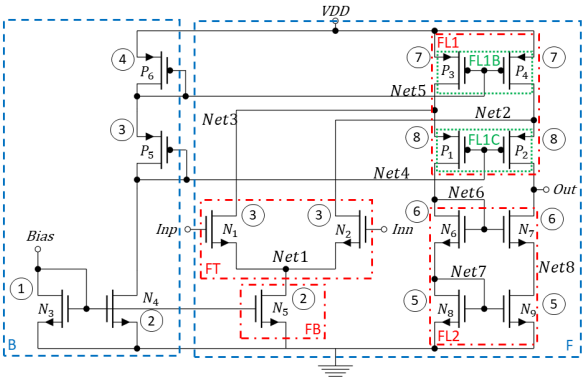

Figure 3: Schematic of a folded-cascode op-amp

Optimization goal: The device dimensions calculated by the tool are optimized with regards to safety margins of performance features:

$$\mathbf{t}(\mathbf{z}) = \mathbf{f}_o, \quad \mathbf{f}_o = [\pm f_1, \pm f_2, ..., \pm f_{n_{f_o}}] \tag{5}$$

$$t_i(\mathbf{z}) = \begin{cases} f_i, & \text{if } f_i \geq f_{B,i} \\ -f_i, & \text{if } f_i \leq f_{B,i} \end{cases} \tag{6}$$

with n_{f_o} as the number of performance features whose safety margins should be maximized.

B. *Circuit Variables and Domains*

Geometries and voltages/currents of the devices form the other circuit variable besides $\mathbf{f}$. For transistors, the geometries consist of channel widths W_k and channel lengths L_k, which are combined in a variable $\mathbf{x}_k$ for all n_T transistors:

$$\mathbf{x} = \{\mathbf{x}_1, \mathbf{x}_2, ..., \mathbf{x}_{n_T}\}, \tag{7}$$

$$\mathbf{x}_k = \begin{pmatrix} W_k \\ L_k \end{pmatrix}, W_k \in D_W, L_k \in D_L, k = 1, 2, ..., n_T. \tag{8}$$

For layout and symmetry reasons, transistor widths and lengths are discretized. The discrete domains D_W and D_L are determined from the process technology for each transistor.

The drain-source currents, the gate-source voltages and the drain-source voltages of the transistors are formulated as:

$$\mathbf{i}_{DS}^{T} = [i_{DS,1}, i_{DS,2}, ..., i_{DS,n_T}], i_{DS,k} \in \mathbb{R}, k = 1, 2, ..., n_T \tag{9}$$

$$\mathbf{v}_{GS}^{T} = [v_{GS,1}, v_{GS,2}, ..., v_{GS,n_T}], v_{GS,k} \in \mathbb{R}, k = 1, 2, ..., n_T \tag{10}$$

$$\mathbf{v}_{DS}^{T} = [v_{DS,1}, v_{DS,2}, ..., v_{DS,n_T}], v_{DS,k} \in \mathbb{R}, k = 1, 2, ..., n_T \tag{11}$$

Notations of variables of other device types as capacitors or resistors hold analogously.

The gate-source voltages and the drain-source voltages are formulated in dependence of the nodal voltages, using Kirchhoff's voltage law (KVL). The voltage variables used in the initial sizing tool are therefore all n_N node voltages $\mathbf{v}_N$:

$$\begin{aligned} &\mathbf{v}_{N}^{T} = [v_{N,1}, v_{N,2}, ..., v_{N,n_N}], v_{N,k} \in \mathbb{R}, k = 1, 2, ..., n_N \\ &[\mathbf{v}_{GS}^{T} \mathbf{v}_{DS}^{T}]^{T} = \mathbf{A} \cdot \mathbf{v}_N, \text{ with } \mathbf{A} \text{ as nodal incidence matrix.} \end{aligned} \tag{12}$$

This is an important difference to the state of the art in initial sizing: Other methods require a manual setup for each individual circuit. The presented initial sizing tool combines the automatism of modified nodal analysis in circuit simulation with a library of analytical performance equations, which allows the automation of the setup for an individual circuit netlist.

C. Kirchhoff's Current Law (KCL)

The specific current flow through a circuit is represented by Kirchhoff's current law for the currents k flowing into node l, for all $n_{\mathcal{N}}$ nets in the individual circuit:

$$\mathbf{c}_{KCL}(\mathbf{i}_{DS}) = \mathbf{0} \;\Leftrightarrow\; \forall_{l \in \mathcal{N}} \sum_{k} i_{DS_k} = 0, |\mathbf{c}_{KCL}| = n_N, \tag{13}$$

These constraints are automatically generated based on an analysis of the circuit netlist. By adding the KCL as constraint to the initial sizing task, no manual current flow inspection is needed. This is another important contribution to an automation of the initial sizing process.

D. Transistors

At this point, the Shichman-Hodges model is implemented to model the transistor behavior [33], which also underlies the manual initial sizing process. To avoid mismatch and dependency of i_{DS} on v_{DS}, transistors classically operate in saturation, which is determined by the following conditions at one transistor k:

$$\begin{aligned} c_{T_{region}}(v_{GS,k}, v_{DS,k}) : v_{GS,k} - v_{th} \geq 0 \wedge \\ v_{DS,k} - (v_{GS,k} - v_{th}) \geq 0 \end{aligned} \tag{14}$$

In saturation, the transistors behave as voltage-controlled current sources, controlled by v_{GS}.

$$\begin{aligned} &c_{T_{behavior}}(i_{DS,k}, v_{GS,k}, v_{DS,k}) : \\ &\quad i_{DS,k} = \frac{\mu C_{ox}}{2} \frac{W_k}{L_k} (v_{GS,k} - v_{th})^2 (1 + \lambda v_{DS,k}) \end{aligned} \tag{15}$$

The behavior and region constraints are automatically generated for all transistors in the circuit.

The transistor behavior is generally described by equality constraints for the drain-source currents $\mathbf{c}_{T_{behavior}}$ and inequality constraints for the operating regions $\mathbf{c}_{T_{region}}$ of all transistors:

$$\mathbf{c}_{T_{behavior}}(\mathbf{i}_{DS}, \mathbf{v}_N, \mathbf{x}) = \mathbf{0} \tag{16}$$

$$\mathbf{c}_{T_{region}}(\mathbf{v}_N) \geq \mathbf{0} \tag{17}$$

Please note that the reformulation of equations (14) and (15) and corresponding equations in dependence of node voltages $\mathbf{v}_N$ according to equation (12) is part of the automatic setup.

E. Transistor Pairings

Analog operational amplifiers consist of basic transistor groups formed by hierarchical transistor pairings, e.g. differential pairs, level shifters, and different types of current mirrors.

An automatic identification for pairings and for symmetry conditions is performed, e.g. [34], [35]. It leads to geometrical conditions for transistor pairings in order to minimize mismatch, e.g., due to channel length modulation or local manufacturing variations:

$$\forall_{pairs\ k,l}\ L_k = L_l \ \wedge\ \forall_{sym.pairs\ k,l}\ W_k = W_l \tag{18}$$

Please note that the implementation of these equations is part of the automatic sizing setup. The matching conditions reduce the number of variables.

F. Functional Circuit Blocks

Transistor pairing represent an intermediate level of abstraction in the composition of an operational amplifier. On the next higher level, an op-amp consists of functional circuit blocks. Functional circuit blocks are, for instance, first to third stage, with respective transconductance, load and bias, and the overall bias block. These functional blocks are formed by different types of transistor pairings. A current mirror is, for instance, part of the load of the first stage in Fig. 2 (N_1, N_2), or part of the bias of the op-amp in Fig. 2 (P_3, P_4).

Specific constraints and intermediate performance expressions emerge for functional circuit blocks. Constraints are, e.g. restrictions on the transistor sizing that ensure proper function of the functional circuit block. For the intermediate performance of a functional circuit block corresponding equations have been established, e.g. [36], [37]. We have gathered them in a library. An automatic analysis of an op-amp netlist for functional circuit blocks has been developed (Sec. III). It is used for an automatic instantiation of the related constraints and intermediate performance expressions. Appendix B gives one example on this topic: the instantiation of the open-loop gain of an op-amp stage as an example for an intermediate performance expression.

In general, the constraints to ensure the proper functionality of the functional circuit blocks are formulated as

$$\mathbf{c}_{FCB,e}(\mathbf{x}, \mathbf{i}_{DS}, \mathbf{v}_N) = \mathbf{0},\ |\mathbf{c}_{FCB,e}| = n_{FCB,e} \tag{19}$$

$$\mathbf{c}_{FCB,i}(\mathbf{x}, \mathbf{i}_{DS}, \mathbf{v}_N) \geq \mathbf{0},\ |\mathbf{c}_{FCB,i}| = n_{FCB,i}. \tag{20}$$

A generalization of the intermediate performance expressions is analogous to the generalization of the op-amp performance constraints (21) and (22) described in the next subsection.

G. Op-Amp Performance

Twelve performance features $\mathbf{f}$ of an operational amplifier (Table II) are calculated by DC operating point parameter values. The equations calculating the performance values are based on [36], [37].

There are two types of constraints for each performance f_k: one is the performance function itself, which is an equality constraint, the other one is the performance specification to ensure that the performance fulfills the given minimum requirement $f_{B,k}$:

$$\mathbf{c}_f(\mathbf{f}, \mathbf{x}, \mathbf{i}_{DS}, \mathbf{v}_N) = \mathbf{0},\ |\mathbf{c}_f| = n_f \quad (21)$$

$$\mathbf{c}_{f_B}(\mathbf{f}, \mathbf{f}_B) \geq \mathbf{0},\ |\mathbf{c}_{f_B}| = n_{f_B} \quad (22)$$

Appendix C shows on the open-loop gain, how these constraints are automatically established.

The formulation of the performance specification as inequality constraints is a key difference to the manual sizing process [9], [36]–[38], which starts from distinct performance values and maps these values back to circuit parameters by an explicit procedure. Using instead mathematical optimization enables the presented initial sizing tool to search a large solution space.

As there are formulas that map from $\mathbf{g}_m, \mathbf{g}_d$ to $\mathbf{i}_{DS}, \mathbf{x}$ and vice versa (Appendix B), these two types of variables are interchangeable in the formulation of the initial sizing problem. In the initial sizing tool, device geometries have been chosen as design parameters, while $\mathbf{g}_m, \mathbf{g}_d$ are computed through the explicit performance functions (33), (34), and (15). Alternatively $\mathbf{g}_m$ and $\mathbf{g}_d$ can be used as design variables. The concept of the automation tool also includes the possibility to add the poles and zeros as design features.

III. Functional Circuit Block Analysis

An outline of the algorithm for functional block analysis is given in Fig. 4. Input are the netlist and the transistor pairings, basically current mirrors (e.g. Fig. 2, N_1, N_2), differential pairs (e.g. Fig.2, P_1, P_2) and inverting stages (e.g. Fig. 2, P_5, N_3). The algorithm outputs the first F and second stage S of the amplifier with its transconductance FT/ST, load FL and bias part FB/SB, and the overall bias B of the amplifier.

First stage: The algorithm starts by identifying the first stage F of the op-amp through the differential pair. It has to be distinguished between differential pairs which are part of the first stage of an op-amp or part of a feedback circuit [38]: If the gate pins of the differential pair are not connected to any other differential pair, the differential pair is the transconductance FT of a first stage.

First stage load: The load FL of the first stage is identified next. There are loads consisting of a single transistor pair, FL, which is the case e.g. for first stages consisting of a simple differential pair and a simple current mirror (Fig. 2). If the first stage consists of a folded-cascode differential pair, which is the case in Fig. 3 (N_1, N_2, P_1, P_2), the load splits into two parts FL1 and FL2, with FL1 containing the cascode pair FL1C of the folded-cascode differential pair and its bias FL1B. A similar partitioning can be used for telescopic op-amps. Here

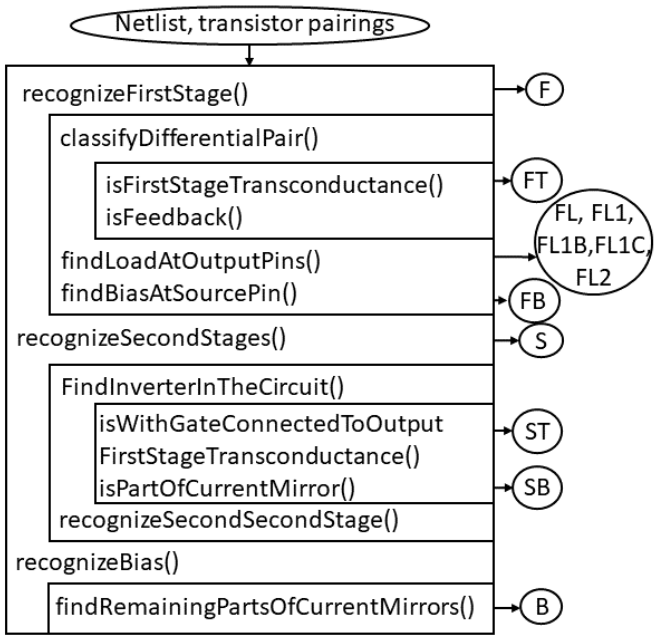

Figure 4: Simplified pseudo-code representation of the functional building block analysis

the cascode pair of the differential pair is the first load part FL1 while the load structure connected to its drain is FL2. The load parts of F are determined by the connections on the drain pins of the differential pair or its cascode version, which are the output pins of F. They are either whole current mirrors (Fig. 2, N_1, N_2) or parts of it (Fig. 3, P_1, P_2, P_3, P_4).

First stage bias: The bias part FB of the first stage is determined by the connection of the source pin of the differential pair. FB needs to be part of a current mirror.

Second stage: A second stage S consists of an inverting stage, e.g., Fig. 2, P_5, N_3, with P_5 of p-type and source connected to the positive supply voltage rail, and with N_3 of n-type and source connected to the negative supply voltage rail or ground. The part of the inverting stage whose gate is connected to the output of the first stage is the second stage transconductance ST. The other part represents the bias of the second stage SB.

Circuit bias: The overall bias B of the amplifier contains the remaining parts of current mirrors of the circuits that are not part of the first or second stage.

Overall algorithm: The algorithm in Fig 4 is a simplified version. The complete algorithm is able to partition symmetrical OTAs into functional blocks: The output inverter is recognized as second stage, its symmetrical match as another second stage. Furthermore, the algorithm can recognize third stages in the same way as second stages. The recognition of a complementary first stage is identical to the recognition of two first stages.

Algorithm 1 Sizing-specific branching heuristic

Require: set of all transistors $\mathcal{T}$
1: $BT := \{\}$ //Initialize set of branched transistors as empty
2: t_{Bias} := findBiasTransistor($\mathcal{T}$)
3: branchTransistors($\{t_{Bias}\}$, BT) //Algorithm 2
4: branchTransistors($\mathcal{T}, BT$) //Branch remaining transistors
5: **if** $|\mathcal{T}| - |BT| == 4$ **then** // Branch cascoded pair transistors
6: t_{bcp} := findBiasOfCascodedPair($\mathcal{T}$)
7: branchTransistor(t_{bcp})
8: t_{cp} := findCascodedPair($\mathcal{T}$)
9: branchTransistor(t_{cp})
10: **end if**

IV. Sizing-specific Branching Heuristic

The following approach on value assignment and branching ordering is essential for a successful sizing process of more complex circuits as e.g. a folded-cascode op-amp. It provides convergence and CPU times of a few seconds.

Value assignment strategy: The overall goals of circuit design are small area and low power consumption. Therefore, the optimization algorithm starts from the lower bounds of transistor variables and searches along increasing values. An exception is the transistor length, where a random value assignment is applied. Thus, the restart of a search has a greater influence on the search behavior (Sec. II).

Variable order strategy:

Branching order of transistor variables: As defined in Sec. II-B, every transistor k has a unique set of variables:

$$t_k = \{\mathbf{x}_k, i_{DS,k}, v_{GS,k}, v_{DS,k}\} \tag{23}$$

Each transistor will be designed as a whole, that means the variables in t_k will be determined together.

Variables with smaller domains should be computed before variables with larger domains within the branching process of constraint programming. Transistor length and width have discrete (i.e., smaller) domains, while $i_{DS,k}, v_{GS,k}, v_{DS,k}$ have float (i.e., larger) domains. For transistor lengths, a smaller value range is browsed than for widths. This suggests the following variable order, which has shown good results in practice:

$$L_k \rightarrow W_k \rightarrow i_{DS,k} \rightarrow v_{GS,k} \tag{24}$$

$v_{DS,k}$ is skipped here. It has a minor effect (channel length modulation) in (15) and in the region constraints (17), and does not provide sufficient guidance in an early stage of branching.

Algorithm for the branching order of transistors in a circuit: The branching ordering for all transistors is computed by the Algorithms 1 and 2. The results for the two example circuits are indicated by circled numbers in Figs. 2 and 3.

The core idea is to follow the signal path of the biasing input through the circuit. Afterwards, the algorithm branches over the remaining transistors (Algorithm 1, Line 4).

The algorithms are based on two rules:

Algorithm 2 branchTransistors

Require: set of transistors $\mathcal{T}$, set of transistors BT branched on
1: **if not** $\mathcal{T} == \{\}$ **then**
2: $\mathcal{T}_c := \{\}$ //Initialize set of connected transistors as empty
3: **for all** t in $\mathcal{T}$ **do**
4: **if not** isCascodedPair(t) **and not** isBiasOfCascodedPair(t) **and** $t \notin BT$ **then**
5: branchTransistor(t)
6: $BT := BT \cup \{t\}$
7: $\mathcal{T}_c$:= $\mathcal{T}_c \cup$ findConnectedTransistors(t, BT)
8: **end if**
9: **end for**
10: branchTransistors($\mathcal{T}_c, BT$) //Recursive call
11: **end if**

1) Branch first over transistors with smaller variable domains to avoid so-called wrong turns to "dead solution ends" in the branching process.
2) Branch over transistors that are connected to transistors which have already been branched over. This is concentrating the search on feasible and promising regions.

The bias current i_{Bias} is given as input to the sizing task (Sec. II-A). This is used to start sizing the transistor whose drain is fed with i_{Bias} (Algorithm 1, Line 2 - 3). With given i_{Bias} and (15), the size of the domains of the variables of the bias transistor t_{bias} is significantly reduced. For the two-stage op-amp, $bias = P_3$, for the folded- cascode op-amp, $bias = N_3$. By the bias transistor as part of a current mirror, also the gate-source voltages of the transistors connected to the bias transistor gates are set, e.g. for the two-stage op-amp $v_{GS,P_3} = v_{GS,P_4} = v_{GS,P_5}$, which in turn reduces their respective domain sizes. This suggests to branch over these transistors next.

For the two-stage op-amp, after assigning the variables of t_{P_4} and t_{P_5}, the currents of the transconductances of the first and second stage ($i_{DS,P_1}, i_{DS,P_2}, i_{DS,N_3}$) are determined according to (13). Hence, the algorithm branches over them next, propagating branching through the circuit. With findConnectedTransistors() (Algorithm 2, Line 7), all transistors $\mathcal{T}_c$ are computed that are connected to a transistor t_k whose variables have just been branched over.

A particular case are transistors on the bias path that play an important role in the overall circuit function. These transistors are branched at a later point in the branching ordering. An example is the cascode pair, Fig. 3, (P_1, P_2), and its bias (P_3, P_4). The transistors of the cascode pair depend on a large number of other transistors (N_1, N_2, P_3, P_4, N_6, N_7), and have a great impact on the behavior of the overall circuit. It is ensured, that it is not branched over these four transistors randomly (Algorithm 2, Line 4). Instead, if the first stage is a cascode one, the branching of the load, containing the cascode pair, takes place in the last step of the algorithm (Algorithm 1,

Variable	Value
$W_{P_1} = W_{P_2}$	21μm
W_{P_3}	11μm
W_{P_4}	21μm
W_{P_5}	112μm
$W_{N_1} = W_{N_2}$	10μm
W_{N_3}	21μm
$L_{P_1} = L_{P_2}$	2μm
$L_{P_3} = L_{P_4} = L_{P_5}$	3μm
$L_{N_1} = L_{N_2}$	5μm
L_{N_3}	1μm
C_c	4.5pF

(a) Two-stage op-amp

Variable	Value
$W_{N_1} = W_{N_2}$	116μm
W_{N_3}	9μm
W_{N_4}	9μm
W_{N_5}	26μm
$W_{N_6} = W_{N_7} = W_{N_8} = W_{N_9}$	85μm
$W_{P_1} = W_{P_2} = W_{P_3} = W_{P_4}$	95μm
$W_{P_5} = W_{P_6}$	22μm
$L_{N_1} = L_{N_2}$	9μm
$L_{N_3} = L_{N_4} = L_{N_5}$	9μm
$L_{N_6} = L_{N_7} = L_{N_8} = L_{N_9}$	1μm
$L_{P_1} = L_{P_2} = L_{P_3} = L_{P_4} = L_{P_5} = L_{P_6}$	2μm

(b) Folded-cascode op-amp

Table I: Initial dimensions

Constraints	Spec.		Initial sizing tool		BSIM3v3	
	(a)	(b)	(a)	(b)	(a)	(b)
Max. output voltage (V)	≥ 4.6	≥ 3.5	4.6	3.7	4.8	3.5
Min. output voltage (V)	≤ 0.4	≤ 0.4	0.2	0.4	0.2	0.7
Max. common-mode input voltage (V)	≥ 3.5	≥ 3.5	3.8	4.7	4.3	3.4
Min. common-mode input voltage (V)	≤ 1	≤ 2	0.1	1.9	0.2	0.81
Quiescent power (mW)	≤ 1	≤ 10	0.66	5.3	0.68	5.4
Open-loop gain (dB)	≥ 80	≥ 65	90	73	84	74
Unity-gain bandwidth (MHz)	≥ 2.5	≥ 2.5	3	6.4	2.1	6.7
Phase Margin ($^\circ$)	≥ 60	≥ 60	60	88	66	85
Slew rate ($\frac{V}{\mu s}$)	≥ 3.5	≥ 3.5	4.2	14.3	3.9	10.8
CMRR (dB)	≥ 80	≥ 65	97	127	98	138
Neg. PSRR (dB)	≥ 90	≥ 70	95	-	100.5	79
Pos. PSRR (dB)	≥ 80	≥ 70	138	-	100	82

Table II: Performance values of (a) two-stage op-amp (b) folded-cascode op-amp

Tool/Author	Setup time	Execution time
GPCAD [6]	**Long**	Few sec
IDAC [1]	**Months**	Few sec
OASYS [3]	6 **months**	3 sec
BLADES [2]	Long	20 min
MINLP [41]	6 **months**	1 min
Here	Few **sec**	Few sec

Table III: Comparison setup and execution time of state-of-the-art equation-based automatic sizing tools [40] with the new developed method

Lines 5-10).

Branching over $\mathbf{v}_{DS}$ of all transistors concludes the branching order.

Adding constraints to the initial sizing problem changes the search behavior. Although a transistor based branching showed the best results, further investigations are underway, e.g. considering performance requirements in the branching strategy.

V. Experimental Results

The presented initial sizing tool, COPRISI (COnstraint-PRogrammed Initial SIzing), was verified using 16 different topologies. This paper presents results for a two-stage op-amp (Fig. 2) and a folded-cascode op-amp (Fig. 3) with a 0.25μm PDK, a supply voltage of 5V, and a driving load of 20pF. The input bias current was 10μA for the two-stage op-amp, and 100μA for the folded-cascode op-amp. The domains of the transistor dimensions were $D_L = [1, ..., 10]$ μm, $D_W = [1, ..., 600]$ μm with a discretization of 1 μm. The performance specifications are given in Table II. The results of the hierarchical analysis of the two op-amps according to Sec. III are marked in color in Figs. 2 and 3.

For every topology, a few seconds are needed to set up the design problem and to solve the constraint programming problem. It provides sets of sizing solutions satisfying the specifications in Table II under the constraint of a maximum total gate area of 6200μm^2. Table 1a and 1b give selected sizings for maximum slew rate and unity-gain bandwidth.

The performance values resulting from the analytical formulas within the initial sizing tool have been compared to simulation using the BSIM3v3 transistor model. Table II shows the results[1]. For most of the performance features, the results of the automation tool are in good agreement with SPICE level simulation. The obtained deviations correspond to the expectation of designers.

Table III compares the setup and execution time of state-of-the-art equation-based automatic sizing tool with the newly developed initial sizing tool. The values are taken from [40]. The presented initial sizing tool stands out for its small setup time. The reason for the long setup times of the state-of-the-art tools is, that the design plans for every new topology have to be set up manually. This step is automated in the new tool. Please note that this paper is the first new contribution since [31] on initial, equation-based sizing.

[1]PSRR was not considered for the folded-cascode op-amp, as no good analytical formula is available due to many possible noise sources [39].

VI. Conclusion

This paper presented a new method for automated initial sizing of analog circuits. It complies with designers' request for interpretable, physics-based formulas. A new approach to automatically set up the design problem has been presented. It features an analysis of the hierarchical composition of an op-amp. The set-up effort reduces from months to seconds in this way. The optimization problem is solved using constraint programming with a new circuit-related branching heuristic. The method was verified using 16 different op-amp topologies. Future work will be on considering feedback loops, parameter statistics and transistors in weak/moderate or strong inversion.

References

[1] M. G. R. Degrauwe, O. Nys, E. Dijkstra, J. Rijmenants, S. Bitz, B. L. A. G. Goffart, E. A. Vittoz, S. Cserveny, C. Meixenberger, G. van der Stappen, and H. J. Oguey, "IDAC: An interactive design tool for analog CMOS circuits," *IEEE Journal of Solid-State Circuits*, 1987.

[2] F. El-Turky and E. Perry, "BLADES: An artificial intelligence approach to analog circuit design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 1989.
[3] R. Harjani, R. A. Rutenbar, and L. Carley, "OASYS: A Framework for Analog Circuit Synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 1989.
[4] H. Y. Koh, C. H. Sequin, and P. R. Gray, "OPASYN: a compiler for CMOS operational amplifiers," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 1990.
[5] F. Leyn, W. Daems, G. Gielen, and W. Sansen, "Analog Circuit Sizing with Constraint Programming Modeling and Minimax Optimization," in *IEEE International Symposium on Circuits and Systems*, 1997.
[6] M. del Mar Hershenson, S. P. Boyd, and T. H. Lee, "GPCAD: a tool for CMOS op-amp synthesis," in *1998 IEEE/ACM International Conference on Computer-Aided Design*, Nov 1998.
[7] G. Van der Plas, G. Debyser, F. Leyn, K. Lampaert, J. Vandenbussche, G. Gielen, W. Sansen, P. Veselinovic, and D. Leenaerts, "AMGIE–A Synthesis Environment for CMOS Analog Integrated Circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2001.
[8] P. C. Maulik and L. R. Carley, "Automating analog circuit design using constrained optimization techniques," in *IEEE/ACM International Conference on Computer-Aided Design*, 1991.
[9] D. M. Binkley, C. E. Hopper, S. D. Tucker, B. C. Moss, J. M. Rochelle, and D. P. Foty, "A CAD methodology for optimizing transistor current and sizing in analog CMOS design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2003.
[10] C.-J. R. Shi and X.-D. Tan, "Canonical Symbolic Analysis of Large Analog Circuits with Determinant Decision Diagrams," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2000.
[11] W. Verhaegen and G. Gielen, "Efficient DDD-based Symbolic Analysis of Large Linear Analog Circuits," in *ACM/IEEE Design Automation Conference*, 2001.
[12] X.-X. Liu, A. A. Palma-Rodriguez, S. Rodriguez-Chavez, S. X.-D. Tan, E. Tlelo-Cuautle, and Y. Cai, "Performance bound and yield analysis for analog circuits under process variations," in *Asia and South Pacific Design Automation Conference*, 2013.
[13] W. Nye, D. C. Riley, A. Sangiovanni-Vincentelli, and A. L. Tits, "DELIGHT.SPICE: an optimization-based system for the design of integrated circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 1988.
[14] K. Antreich and H. Graeb, "Circuit optimization driven by worst-case distances," in *IEEE/ACM International Conference on Computer-Aided Design*, 1991.
[15] K. Antreich, H. Graeb, and C. Wieser, "Circuit analysis and optimization driven by worst-case distances," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 1994.
[16] E. S. Ochotta, R. A. Rutenbar, and L. R. Carley, "Synthesis of High-Performance Analog Circuits in ASTRX/OBLX," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 1996.
[17] R. Phelps, M. Krasnicki, R. A. Rutenbar, L. R. Carley, and J. R. Hellums, "Anaconda: Simulation-Based Synthesis of Analog Circuits Via Stochastic Pattern Search," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2000.
[18] B. Liu, F. V. Fernandez, and G. Gielen, "Efficient and Accurate Statistical Analog Yield Optimization and Variation-Aware Circuit Sizing Based on Computational Intelligence Techniques," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2011.
[19] M. Eick and H. E. Graeb, "MARS: Matching-Driven Analog Sizing," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2012.
[20] F. Gong, H. Yu, Y. Shi, and L. He, "Variability-Aware Parametric Yield Estimation for Analog/Mixed-Signal Circuits: Concepts, Algorithms, and Challenges," *IEEE Design and Test*, 2014.
[21] G. Berkol, E. Afacan, G. Dundar, A. E. Pusane, and F. Baskaya, "A novel yield aware multi-objective analog circuit optimization tool," in *IEEE International Symposium on Circuits and Systems*, 2015.
[22] M. Meissner and L. Hedrich, "FEATS: Framework for Explorative Analog Topology Synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2015.
[23] F. Passos, E. Roca, J. Sieiro, R. Fiorelli, R. Castro-Lopez, J. M. Lopez-Villegas, and F. V. Fernandez, "A Multilevel Bottom-up Optimization Methodology for the Automated Synthesis of RF Systems," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2019.
[24] A. Canelas, R. Povoa, R. Martins, N. Lourenco, J. Guilherme, J. P. Carvalho, and N. Horta, "FUZYE: A Fuzzy C-Means Analog IC Yield Optimization using Evolutionary-based Algorithms," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2019.
[25] *WiCkeD*, www.muneda.com, MunEDA, 2009. [Online]. Available: www.muneda.com
[26] C. Schulte, G. Tack, and M. Z. Lagerkvist, "Modeling and Programming with Gecode," https://www.gecode.org/doc-latest/MPG.pdf, 2019.
[27] G. Tack, C. Schulte, and G. Smolka, "Generating propagators for finite set constraints," in *Proceedings of the 12th International Conference on Principles and Practice of Constraint Programming*, F. Benhamou, Ed. Springer, 2006.
[28] T. Fruhwirth and S. Abdennadher, *Essentials of Constraint Programming*. Berlin, Heidelberg: Springer-Verlag, 2003.
[29] P. Hofstedt, *Multiparadigm Constraint Programming Languages*. Springer, 2011.
[30] I. P. Gent, C. Jefferson, and I. Miguel, "MINION: A Fast, Scalable, Constraint Solver," in *17th European Conference on Artificial Intelligence ECAI*, 2006.
[31] M. d. Hershenson, S. P. Boyd, and T. H. Lee, "Optimal design of a CMOS op-amp via geometric programming," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2001.
[32] M. Z. L. Christian Schulte, Guido Tack, *Modeling and Programming with Gecode*, 6th ed., www.gecode.org, May 2018.
[33] H. Shichman and D. A. Hodges, "Modeling and simulation of insulated-gate field-effect transistor switching circuits," *IEEE Journal of Solid-State Circuits SC*, 1968.
[34] T. Massier, H. Graeb, and U. Schlichtmann, "The Sizing Rules Method for CMOS and Bipolar Analog Integrated Circuit Synthesis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2008.
[35] H. Graeb, S. Zizala, J. Eckmueller, and K. Antreich, "The sizing rules method for analog integrated circuit design," in *IEEE/ACM International Conference on Computer Aided Design.*, 2001.
[36] K. R. Laker and W. M. C. Sansen, *Design of analog integrated circuits and systems*. McGraw-Hill, 1994.
[37] D. R. H. Phillip E. Allan, *CMOS Analog Circuit Design*. Oxford University Press, 2012.
[38] K. M. David Johns, *Analog Integrated Circuit Design*. John Wiley and Sons, 1997.
[39] D. Stefanovic and M. Kayal, *Structured Analog CMOS Design*. Springer Netherlands, 2008.
[40] N. Lourenco, R. Martins, and N. C. G. Horta, *Automatic Analog IC Sizing and Optimization Constrained with PVT Corners and Layout Effects*. Springer, 2017.
[41] P. C. Maulik, L. R. Carley, and R. A. Rutenbar, "Integer programming based topology selection of cell-level analog circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 14, no. 4, pp. 401–412, April 1995.
[42] T. W. Francesca Rossi, Peter van Beek, *Handbook of Constraint Programming*, 2nd ed. Elsevier, 2006, vol. Volume 2.
[43] M. Zwerger and H. Graeb, "Short-circuit-path and floating-node verification of analog circuits in power-down mode," in *2012 International Conference on Synthesis, Modeling, Analysis and Simulation Methods and Applications to Circuit Design (SMACD)*, Sep. 2012, pp. 241–244.

APPENDIX A
CONSTRAINT PROGRAMMING

Constraint Programming [42] is a programming paradigm which allows to model a mathematical problem with a set of variables $\mathbf{z}$, a domain D_i for for each variable $z_i \in \mathbf{z}$ and a set of constraints C:

$$\mathbf{z} = \{z_1, z_2, ..., z_n\}, \quad z_i \in D_i, \ i \in [1, .., n] \quad (25)$$

$$C = E \cup I = \{c_1, c_2, ..., c_m\} \quad (26)$$

The domains D_i contain either boolean, integer or float values and are either finite or infinite. A constraint $c_j \in E$ is

formulated as equality constraint, i.e. $c_j(\mathbf{z}) = 0$, a constraint $c_k \in I$ as inequality constraint, i.e. $c_j(\mathbf{z}) \bigcirc 0$, with $\bigcirc \in \{>, <, \geq, \leq, !=, ...\}$. Without loss of generality, inequality constraints are formulated with "$\geq$" in the following. The constraints $c \in C$ form a constraint satisfaction problem CSP:

$$CSP(\mathbf{z}) \Leftrightarrow \forall_{j \in E} c_j(\mathbf{z}) = 0 \wedge \forall_{k \in I} c_k(\mathbf{z}) \geq 0, \tag{27}$$

which must be fulfilled. A constraint solver computes all solutions, i.e. value assignments of $\mathbf{z}$, which fulfill the CSP.

If an additional vector of target functions $\mathbf{t}(\mathbf{z})$ is given, a constraint optimization problem (COP) can be formulated as

$$\max \quad \mathbf{t}(\mathbf{z}) \quad s.t. \quad CSP(\mathbf{z}) \Leftrightarrow 1. \tag{28}$$

Solving (28) yields the maximum solution of $\mathbf{t}(\mathbf{z})$ while satisfying $CSP(\mathbf{z})$.

Constraint optimization problems can be solved by methods based on branch-and-bound (BAB) algorithms. The solution space of the COP is explored by branching, i.e. by subsequently splitting the variable domains D into smaller intervals, until the determined value assignment can be accepted or rejected as a solution of the COP. During that process, constraints are analyzed to shrink the domain sizes of the variables: domain values of z_i which cannot fulfill a constraint $c \in C$ are excluded from D_i. The bound step cuts away subspaces which do not lead to better solutions than the ones found so far.

Appendix B
Constraints Derived of Functional Blocks in Operational Amplifiers

The following section gives one example of constraints of functional blocks: the derivation of intermediate performance expressions. It shows, how the specific equations for different topologies can be derived from generalized equations using the automatic functional block analysis (Sec. III).

The open-loop gain of an amplification stage j is an example for an intermediate performance expression of an op-amp. It is defined by the product of the transconductance and output resistance of j:

$$A_{D0,j} = g_{m,j} \cdot R_{out,j} \tag{29}$$

The transconductance is defined by the input functional block of the stage, e.g. in Figs. 2 and 3 by the transistors of FT and ST, respectively. $R_{out,j}$ is defined by the functional blocks connected to the output of the stage:

$$R_{out,n} = \frac{1}{g_{p,j} + g_{n,j}}, \tag{30}$$

where $g_{p,j}$ is the output conductance of the functional block connected to the output of the stage j and p-type and $g_{n,j}$ the functional block connected to the output and n-type.

Following this schema, for the two-stage op-amp in Fig. 2, we obtain for the open-loop gain of it stages:

$$A_{D0,1} = \frac{g_{m,P_1}}{g_{d,P_1} + g_{d,N_1}}, \; A_{D0,2} = \frac{g_{m,N_3}}{g_{d,N_3} + g_{d,P_5}} \tag{31}$$

For the folded-cascode op-amp in Fig. 3, we obtain

$$A_{D0,1} = \frac{g_{m,N_1}}{\frac{g_{d,P_2}(g_{d,P_4} + g_{d,N_2})}{g_{m,P_2}} + \frac{g_{d,N_9} g_{d,N_7}}{g_{m,N_7}}} \tag{32}$$

In these formulas, the transconductance $g_{m,k} = \frac{\partial i_{DS,k}}{\partial v_{GS,k}}$ and the output conductance $g_{d,k} = \frac{\partial i_{DS,k}}{\partial v_{DS,k}}$ of a transistor k are required. They are calculated by the respective derivation of the transistor drain-source current $i_{DS,k}$ according to (15):

$$g_{m,k} = \sqrt{2 \mu_k C_{ox} \frac{W_k}{L_k} |i_{DS,k}|} \tag{33}$$

$$g_{d,k} = \lambda |i_{DS,k}| \tag{34}$$

Appendix C
Performance Constraints of Operational Amplifiers

The following section gives one example on performance constraints of operational amplifiers: the open-loop gain. The general performance expressions are stated. It is shown, how they can be specified for different topologies.

The open-loop gain of an operational amplifier is defined by the multiplication of the open-loop gain of its stages:

$$f_{A_{D0}} = \prod_{k=0}^{n_{stages}} A_{D0_i} \tag{35}$$

The open-loop gains of the individual stages can be calculated by its functional blocks as shown in Appendix B. For the two example amplifiers in Figs. 2 and 3, the amplification of each stage is given in (31) and (32), respectively.

As stated in Sec. II-G, two constraint must be posted for the open-loop gain in the constraint program. An equality constraint to calculate the gain (35) and an inequality constraint to ensure that the minimum required gain $f_{B,A_{D0}}$ is fulfilled:

$$f_{A_{D0}} \geq f_{B,A_{D0}} \tag{36}$$